

**Министерство науки и высшего образования Российской  
Федерации ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ**

**ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6**

**«Работа с БД в СУБД MongoDB»**

**по дисциплине «Проектирование и реализация баз данных»**

**Обучающийся** Голинский Данила Владимирович

**Факультет** прикладной информатики

**Группа** K3240

**Направление подготовки** 09.03.03 Прикладная информатика

**Образовательная программа** Мобильные и сетевые технологии

**2025 Преподаватель** Говорова Марина Михайловна

Санкт-  
Петербург  
2026/2027

## Цель работы

Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегацией и изменением данных, ссылками и индексами в базе данных MongoDB.

## Практическое задание

В ходе лабораторной работы необходимо создать базу данных learn, заполнить коллекцию unicorns документами, выполнить запросы выборки, логические условия, работу с массивами, вложенными объектами, курсорами, агрегированными запросами, операциями изменения и удаления данных, ссылками между коллекциями, индексами и анализом плана запроса.

## Выполнение работы

### Практическое задание 2.1.1. Создание БД learn и заполнение коллекции unicorns

#### Команды:

JavaScript

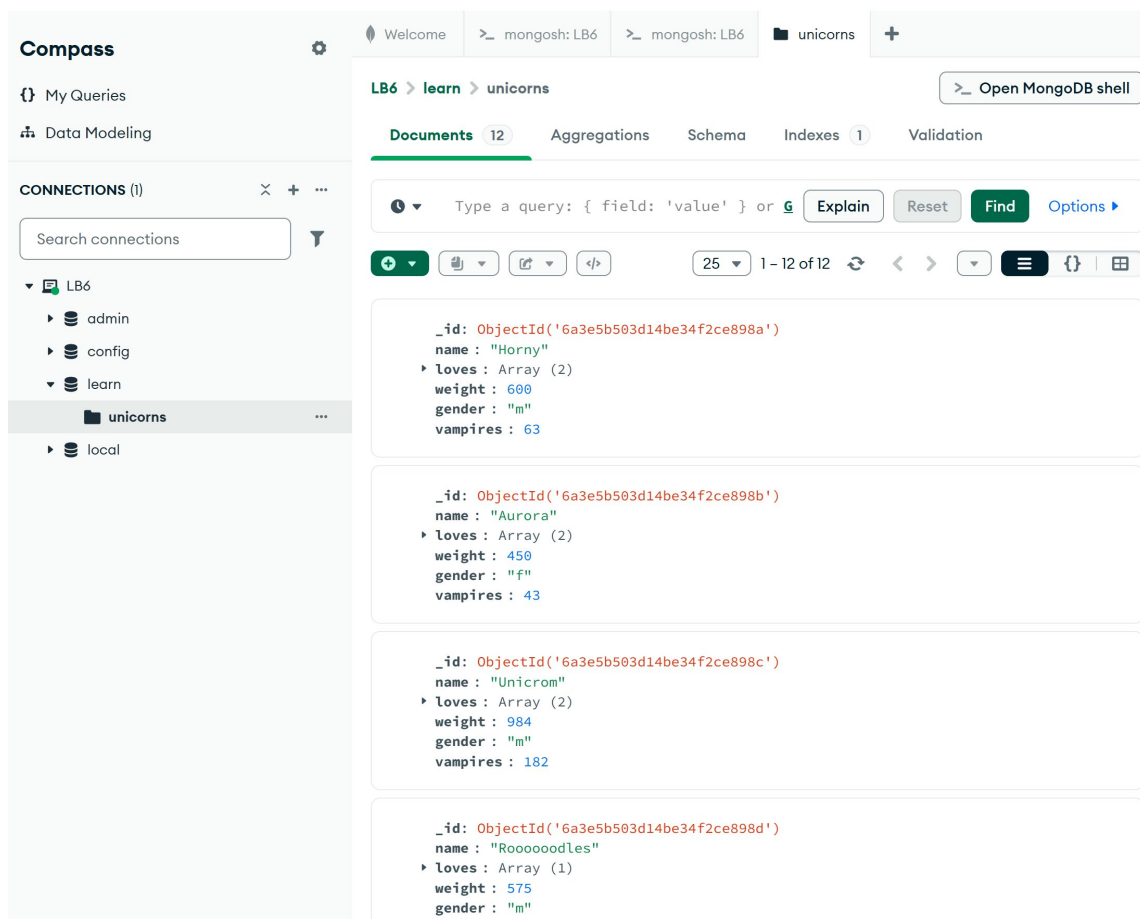
use learn

```
db.unicorns.deleteMany({})
```

```
db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
  {name: 'Rooodoodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}
])
```

```
const document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
db.unicorns.insertOne(document)
```

```
db.unicorns.find()
```



1: Вывод

всех документов из коллекции unicorns в Compass или mongosh]

**Вывод:** В результате создана база данных learn, коллекция unicorns успешно заполнена и содержит 12 документов.

### Практическое задание 2.2.1. Выборка самцов и самок

#### Команды:

##### JavaScript

```
// Выборка самцов с сортировкой по имени  
db.unicorns.find({gender: 'm'}).sort({name: 1})
```

```
// Выборка первых трех самок с сортировкой по имени  
db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
```

```
> db.unicorns.find({gender: 'm'}).sort({name: 1})
< {
  _id: ObjectId('6a3e5b5c3d14be34f2ce8995'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a3e5b503d14be34f2ce898a'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
```

самцов с сортировкой]

2: Результат выборки

```

> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
< {
  _id: ObjectId('6a3e5b503d14be34f2ce898b'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a3e5b503d14be34f2ce898f'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 733,
  gender: 'f',
  vampires: 40
}
{
  _id: ObjectId('6a3e5b503d14be34f2ce8992'),
  name: 'Leia',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 600,
  gender: 'f',
  vampires: 35
}
]

```

3: Результат

выборки первых трех самок]

**Вывод:** Запросы позволяют получить отсортированный список самцов и ограниченный список первых трех самок по заданному критерию пола.

**Практическое задание 2.2.1 (продолжение). Поиск самок, предпочитающих carrot**

**Команды:**

## JavaScript

```
db.unicorns.find({gender: 'f', loves: 'carrot'})
```

```
db.unicorns.findOne({gender: 'f', loves: 'carrot'})
```

```
db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
```

```
> db.unicorns.find({gender: 'f', loves: 'carrot'})
< {
  _id: ObjectId('6a3e5b503d14be34f2ce898b'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a3e5b503d14be34f2ce898e'),
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80
}
```

морковь]

4: Поиск всех самок, любящих

```
> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
< {
  _id: ObjectId('6a3e5b503d14be34f2ce898b'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn>
```

выполнения findOne]

5: Результат

```
> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
< {
  _id: ObjectId('6a3e5b503d14be34f2ce898b'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn>
```

6: Результат выполнения find с limit(1)]

**Вывод:** Методы find, findOne и limit позволяют гибко управлять объемом выдачи: получать либо все подходящие документы, либо строго один.

### Практическое задание 2.2.2. Исключение полей из результата

## Команды:

JavaScript

```
db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0}).sort({name: 1})
```

```
db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0}).sort({name: 1})
{
  _id: ObjectId('6a3e5b5c3d14be34f2ce8995'),
  name: 'Dunx',
  weight: 704,
  vampires: 165
}
{
  _id: ObjectId('6a3e5b503d14be34f2ce898a'),
  name: 'Horny',
  weight: 600,
  vampires: 63
}
{
```

7: Вывод списка самцов без полей loves и gender]

**Вывод:** Проекция документов позволяет скрывать ненужные свойства. В результате у документов отсутствуют поля loves и gender, но выводятся name, weight и vampires.

## Практическое задание 2.2.3. Обратный порядок добавления

## Команды:

JavaScript

```
db.unicorns.find().sort({$natural: -1})
```



```

> db.unicorns.find().sort({$natural: -1})
< {
  _id: ObjectId('6a3e5b5c3d14be34f2ce8995'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a3e5b503d14be34f2ce8994'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],

```

8: Вывод списка

единорогов в обратном порядке]

**Вывод:** Использование мета-поля и параметра `$natural: -1` возвращает документы в порядке, обратном их физическому добавлению в коллекцию.

#### Практическое задание 2.1.4. Первый элемент массива loves

##### Команды:

JavaScript

```
db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
```

```
> db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
< {
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{
  name: 'Aurora',
  loves: [
    'carrot'
  ]
}
{
  name: 'Unicrom',
  loves: [
    'energon'
  ]
}
```

9: Проекция с

оператором \$slice для массива loves]

**Вывод:** Оператор \$slice успешно ограничивает массив, выводя только первое предпочтение единорогов. Служебное поле \_id при этом скрыто.

### Практическое задание 2.3.1. Самки весом от 500 до 700 кг

#### Команды:

JavaScript

```
db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
```

```

> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
< {
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80
}
{
  name: 'Leia',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 601,
  gender: 'f',
  vampires: 33
}

```

10: Выборка самок по диапазону веса]

**Вывод:** Использование условных операторов сравнения \$gte (больше или равно) и \$lte (меньше или равно) позволяет эффективно осуществлять выборку по числовым диапазонам.

### Практическое задание 2.3.2. Самцы весом от 500 кг, предпочитающие grape и lemon

#### Команды:

JavaScript

```

db.unicorns.find({
  gender: 'm',
  weight: {$gte: 500},
  loves: {$all: ['grape', 'lemon']}
}, {_id: 0})

```

```

> db.unicorns.find({
  gender: 'm',
  weight: {$gte: 500},
  loves: {$all: ['grape', 'lemon']}}
}, {_id: 0})
< {
  name: 'Kenny',
  loves: [
    'grape',
    'lemon'
  ],
  weight: 690,
  gender: 'm',
  vampires: 39
}

```

11:

Выборка самцов с использованием оператора \$all]

**Вывод:** Оператор \$all осуществляет строгую проверку того, что массив loves содержит в себе переданный перечень элементов одновременно.

### Практическое задание 2.3.3. Документы без поля vampires

#### Команды:

JavaScript

```
db.unicorns.find({vampires: {$exists: false}}, {_id: 0})
```

```

> db.unicorns.find({vampires: {$exists: false}}, {_id: 0})
< {
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}

```

12:

Поиск документов без поля vampires]

**Вывод:** Логический оператор `$exists: false` нашел документ `Nimue`, у которого поле подсчета побежденных вампиров изначально отсутствовало в схеме.

### Практическое задание 2.3.4. Самцы с первым предпочтением

#### Команды:

JavaScript

```
db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
```

```

> db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
< {
  name: 'Dunx',
  loves: [
    'grape'
  ]
}
{
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{

```

13: Упорядоченный список самцов с первым предпочтением]

**Вывод:** Комбинация проекции, фильтрации и сортировки вывела только имена самцов и их главное (первое) предпочтение из массива.

### Практическое задание 3.1.1. Создание towns с вложенным объектом mayor

#### Команды:

JavaScript

```
db.towns.drop()
```

```
db.towns.insertMany([
  {name: "Punxsutawney", population: 6200, last_sensus: ISODate("2008-01-31"), famous_for: ["phil the groundhog"],
  mayor: { name: "Jim Wehrle" }},
  {name: "New York", population: 22200000, last_sensus: ISODate("2009-07-31"), famous_for: ["statue of liberty",
  "food"], mayor: { name: "Michael Bloomberg", party: "I"}},
  {name: "Portland", population: 528000, last_sensus: ISODate("2009-07-20"), famous_for: ["beer", "food"], mayor:
  { name: "Sam Adams", party: "D"}}
])
```

```
db.towns.find()
```

```

{
  _id: ObjectId('6a3e5f2f3d14be34f2ce8996'),
  name: 'Punxsutawney',
  population: 6200,
  last_sensus: 2008-01-31T00:00:00.000Z,
  famous_for: [
    'phil the groundhog'
  ],
  mayor: {
    name: 'Jim Wehrle'
  }
}
{
  _id: ObjectId('6a3e5f2f3d14be34f2ce8997'),
  name: 'New York',
  population: 22200000,
  last_sensus: 2009-07-31T00:00:00.000Z,
  famous_for: [
    'statue of liberty',
    'food'
  ],
  mayor: {
    name: 'Michael Bloomberg',
    party: 'I'
  }
}
{
  _id: ObjectId('6a3e5f2f3d14be34f2ce8998'),
  name: 'Portland',
  population: 528000,

```

14: Создание и вывод

коллекции городов с вложенными структурами]

**Вывод:** Создана коллекция towns, демонстрирующая денормализованную структуру данных, где информация о мэре представлена вложенным объектом mayor.

### Практическое задание 3.1.1 (продолжение). Запросы к вложенному объекту mayor

#### Команды:

JavaScript

// Поиск города с независимым мэром (party: 'I')

```
db.towns.find({'mayor.party': 'I'}, {_id: 0, name: 1, mayor: 1})
```

// Поиск города, где у мэра нет поля party

```
db.towns.find({'mayor.party': {'$exists': false}}, {_id: 0, name: 1, mayor: 1})
```

```
> db.towns.find({'mayor.party': 'I'}, {_id: 0, name: 1, mayor: 1})
{
  name: 'New York',
  mayor: {
    name: 'Michael Bloomberg',
    party: 'I'
  }
}
```

15: Поиск города по вложенному свойству party]

```
> db.towns.find({'mayor.party': {'$exists': false}}, {_id: 0, name: 1, mayor: 1})
< {
  name: 'Punxsutawney',
  mayor: {
    name: 'Jim Wehrle'
  }
}
```

16: Поиск мэра без указанной партии]

**Вывод:** Использование точечной нотации ('mayor.party') позволяет строить полноценные запросы к свойствам вложенных документов.

### Практическое задание 3.1.2. JavaScript-функция и курсор

#### Команды:

##### JavaScript

```
var cursor = db.unicorns.find({ $where: function() { return this.gender === 'm'; } }).sort({name: 1}).limit(2);
```

```
cursor.forEach(function(obj) {
  print(obj.name + ' - ' + obj.gender + ' - ' + obj.weight);
});
```



```
> var cursor = db.unicorns.find({ $where: function() { return this.gender == 'm'; } })

cursor.forEach(function(obj) {
    print(obj.name + ' - ' + obj.gender + ' - ' + obj.weight);
});

< Dunx - m - 704
< Horny - m - 600
```

17: Работа с курсором и итерация через forEach]

**Вывод:** В современной версии MongoDB для вычислений на стороне сервера используется оператор \$where, возвращаемые документы обрабатываются через курсор и метод forEach на стороне клиента.

### Практическое задание 3.2.1. Агрегация: Count

#### Команды:

JavaScript

```
db.unicorns.find({ gender: 'f', weight: {$gte: 500, $lte: 600} }).count()
```

```
> db.unicorns.find({ gender: 'f', weight: {$gte: 500, $lte: 600} }).count()
< 2
```

18: Подсчет количества документов через функцию count]

**Вывод:** Функция count вернула значение 2 (под критерий подошли единороги Solnara и Nimue).

### Практическое задание 3.2.2. Агрегация: Distinct

#### Команды:

JavaScript

```
db.unicorns.distinct('loves')
```

```
> db.unicorns.distinct('loves')
< [
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

19:

Получение уникальных значений через distinct]

**Вывод:** Функция `distinct` собрала уникальный одномерный массив из всех значений, содержащихся во вложенных массивах `loves` всей коллекции.

### Практическое задание 3.2.3. Агрегация: Aggregate

#### Команды:

```
JavaScript
db.unicorns.aggregate([
  { $group: { _id: '$gender', count: { $sum: 1 } } }
])
```

```

> db.unicorns.aggregate([
  { $group: { _id: '$gender', count: {$sum: 1} } }
])
< {
  _id: 'm',
  count: 7
}
{
  _id: 'f',
  count: 5
}

```

20:

Группировка по полу и агрегированный подсчет]

**Вывод:** Фреймворк агрегации через стадию \$group сгруппировал данные по полу и посчитал количество особей: 7 самцов и 5 самок.

### Практическое задание 3.3.1. Добавление Barny

#### Команды:

JavaScript

```
db.unicorns.insertOne({ name: 'Barny', loves: ['grape'], weight: 340, gender: 'm' })
```

```
db.unicorns.find({name: 'Barny'}, {_id: 0})
```

```

> db.unicorns.insertOne({ name: 'Barny', loves: ['grape'], weight: 340, gender: 'm' })

db.unicorns.find({name: 'Barny'}, {_id: 0})
< {
  name: 'Barny',
  loves: [
    'grape'
  ],
  weight: 340,
  gender: 'm'
}

```

21: Вставка документа через insertOne и верификация]

**Вывод:** В актуальных версиях СУБД взамен функции save используется специализированная команда insertOne для гарантированного добавления новой записи.

### Практическое задание 3.3.2. Изменение Айна

#### Команды:

JavaScript

```
db.unicorns.updateOne(  
  {name: 'Ayna'},  
  {$set: {weight: 800, vampires: 51}}  
)
```

```
db.unicorns.find({name: 'Ayna'}, {_id: 0})
```

```
}  
> db.unicorns.updateOne(  
  {name: 'Ayna'},  
  {$set: {weight: 800, vampires: 51}}  
)  
  
db.unicorns.find({name: 'Ayna'}, {_id: 0})  
< {  
  name: 'Ayna',  
  loves: [  
    'strawberry',  
    'lemon'  
  ],  
  weight: 800,  
  gender: 'f',  
  vampires: 51  
}
```

22: Обновление полей weight и vampires у Айна]

**Вывод:** Оператор \$set изменяет исключительно указанные свойства документа, не затрагивая остальные данные.

### Практическое задание 3.3.3. Изменение Raleigh

#### Команды:

JavaScript

```
db.unicorns.updateOne(  
  {name: 'Raleigh'},  
  {$set: {loves: ['redbull']}}  
)
```

```
db.unicorns.find({name: 'Raleigh'}, {_id: 0})
```

```
db.unicorns.find({name: 'Raleigh'}, {_id: 0})  
  
< {  
  name: 'Raleigh',  
  loves: [  
    'redbull'  
  ],  
  weight: 421,  
  gender: 'm',  
  vampires: 2  
}
```

23: Перезапись массива loves у Raleigh]

**Вывод:** При помощи \$set массив loves для Raleigh был успешно перезаписан новым значением ['redbull'].

### Практическое задание 3.3.4. Увеличение vampires у самцов

#### Команды:

JavaScript

```
db.unicorns.updateMany(  
  {gender: 'm'},  
  {$inc: {vampires: 5}}  
)
```

```
db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, gender: 1, vampires: 1}).sort({name: 1})
```

```
{
  name: 'Barney',
  gender: 'm',
  vampires: 5
}
{
  name: 'Dunx',
  gender: 'm',
  vampires: 170
}
{
  name: 'Horny',
  gender: 'm',
  vampires: 68
}
{
  name: 'Kenny',
  gender: 'm',
  vampires: 44
}
{
  name: 'Pilot',
  gender: 'm',
  vampires: 59
}
{
```

24: Массовый инкремент поля vampires у самцов]

**Вывод:** Функция `updateMany` в сочетании с модификатором `$inc` выполнила атомарное увеличение числового значения у всех подходящих документов на 5.

**Практическое задание 3.3.5. Удаление `mayor.party` у `Portland`**

### Команды:

JavaScript

```
db.towns.updateOne(  
  {name: 'Portland'},  
  {$unset: {'mayor.party': 1}}  
)
```

```
db.towns.find({name: 'Portland'}, {_id: 0, name: 1, mayor: 1})
```



```
< {  
  name: 'Portland',  
  mayor: {  
    name: 'Sam Adams'  
  }  
}
```

25: Удаление атрибута из вложенного

объекта города Portland]

**Вывод:** Оператор \$unset удаляет ключ из структуры документа (поле party удалено из вложенного объекта mayor города Portland).

### Практическое задание 3.3.6. Добавление chocolate для Pilot

#### Команды:

JavaScript

```
db.unicorns.updateOne(  
  {name: 'Pilot'},  
  {$push: {loves: 'chocolate'}}  
)
```

```
db.unicorns.find({name: 'Pilot'}, {_id: 0, name: 1, loves: 1})
```

```
< {  
  name: 'Pilot',  
  loves: [  
    'apple',  
    'watermelon',  
    'chocolate'  
  ]  
}
```

26: Аппенд значения в массив через оператор \$push]

**Вывод:** Оператор \$push добавил элемент в конец существующего массива loves без перезаписи остальных его элементов.

### Практическое задание 3.3.7. Добавление sugar и lemon для Aurora

#### Команды:

JavaScript

```
db.unicorns.updateOne(  
  {name: 'Aurora'},  
  {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}  
)  
  
db.unicorns.find({name: 'Aurora'}, {_id: 0, name: 1, loves: 1})
```



```
name: 'Aurora',
loves: [
  'carrot',
  'grape',
  'sugar',
  'lemon'
]
```

\$addToSet]

27: Множественное добавление в массив через

**Вывод:** Использование оператора \$addToSet совместно с модификатором \$each добавило новые элементы в массив как в множество, исключая появление дубликатов.

### Практическое задание 3.4.1. Удаление городов с беспартийными мэрами

#### Команды:

JavaScript

```
db.towns.deleteMany({'mayor.party': {$exists: false}})
```

```
db.towns.find()
```

```

    _id: ObjectId('6a3e5f2f3d14be34f2ce8997'),
    name: 'New York',
    population: 22200000,
    last_sensus: 2009-07-31T00:00:00.000Z,
    famous_for: [
      'statue of liberty',
      'food'
    ],
    mayor: {
      name: 'Michael Bloomberg',
      party: 'I'
    }
  }
}

```

28: Результат удаления

документов из коллекции towns]

**Вывод:** После применения метода deleteMany в коллекции towns остался только город New York, так как у остальных городов поле mayor.party отсутствовало.

### Практическое задание 3.4.1 (продолжение). Очистка коллекции towns и просмотр коллекций

#### Команды:

JavaScript

```
db.towns.deleteMany({})
```

```
db.towns.countDocuments()
```

```
db.getCollectionNames()
```

```

db.getCollectionNames()
< [ 'unicorns', 'towns' ]
learn> |

```

29: Полная очистка towns и вывод списка

коллекций]

**Вывод:** Коллекция towns полностью очищена от записей (количество документов равно 0), но сама она сохранена в базе, что подтверждается системной функцией getCollectionNames().

## Практическое задание 4.1.1. Ссылки на зоны обитания (DBRef)

### Команды:

JavaScript

```
db.habitats.drop()
```

```
db.habitats.insertMany([
  { _id: 'forest', name: 'Magic Forest', description: 'Forest habitat for calm unicorns' },
  { _id: 'mountain', name: 'High Mountains', description: 'Mountain habitat for strong unicorns' },
  { _id: 'lake', name: 'Crystal Lake', description: 'Lake habitat for water-loving unicorns' }
])
```

```
db.unicorns.updateMany( {name: {$in: ['Aurora', 'Nimue', 'Solnara']}}, {$set: {habitat: {$ref: 'habitats', $id: 'forest'}}})
db.unicorns.updateMany( {name: {$in: ['Unicrom', 'Dunx', 'Kenny']}}, {$set: {habitat: {$ref: 'habitats', $id: 'mountain'}}})
db.unicorns.updateMany( {name: {$in: ['Leia', 'Pilot']}}, {$set: {habitat: {$ref: 'habitats', $id: 'lake'}}})
```

```
db.habitats.find()
db.unicorns.find( {habitat: {$exists: true}}, { _id: 0, name: 1, habitat: 1 })
```

```
// Имитация перехода по ссылке:
var aurora = db.unicorns.findOne( {name: 'Aurora'} )
db[aurora.habitat.$ref].findOne( { _id: aurora.habitat.$id })
```

```

    name: 'Aurora',
    habitat: DBRef('habitats', 'forest')
  }
  {
    name: 'Unicrom',
    habitat: DBRef('habitats', 'mountain')
  }
  {
    name: 'Solnara',
    habitat: DBRef('habitats', 'forest')
  }
  {
    name: 'Kenny',
    habitat: DBRef('habitats', 'mountain')
  }
  {
    name: 'Leia',
    habitat: DBRef('habitats', 'lake')
  }
}

```

30: Создание коллекции habitats и добавление структуры DBRef]

```

> var aurora = db.unicorns.findOne({name: 'Aurora'})
  db[aurora.habitat.$ref].findOne({_id: aurora.habitat.$id})

```

31: Получение связанного документа из habitats через ссылку]

**Вывод:** Реализован механизм ручного связывания документов между коллекциями через объектную структуру DBRef (\$ref и \$id). По ссылке можно получить документ зоны обитания. Вывод: в документах unicorns добавлено поле habitat со ссылкой DBRef на коллекцию habitats; по ссылке можно получить документ зоны обитания.

#### Практическое задание 4.2.1. Создание уникального индекса по полю name

##### Команды:

JavaScript

```
db.unicorns.createIndex({name: 1}, {unique: true})
```

```
db.unicorns.getIndexes()
```

// Проверка ограничения уникальности (попытка вставки дубликата):

```
db.unicorns.insertOne({name: 'Aurora', loves: ['apple'], weight: 500, gender: 'f'})
```

```
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

```
> db.unicorns.insertOne({name: 'Aurora', loves: ['apple'], weight: 500, gender: 'f'})
```

```
✖ ▶ MongoServerError: E11000 duplicate key error collection: learn.unicorns index: nan
32: Ошибка Duplicate Key при попытке повторной вставки Aurora]
```

**Вывод:** Уникальный индекс успешно построен, так как текущие имена уникальны. При попытке повторно вставить документ с name: 'Aurora' СУБД возвращает ошибку дублирования ключа, гарантируя целостность данных.

### Практическое задание 4.3.1. Управление индексами

#### Команды:

JavaScript

```
db.unicorns.getIndexes()
```

```
db.unicorns.dropIndexes() // Удаление пользовательских индексов
```

```
db.unicorns.getIndexes()
```

// Попытка удалить системный индекс по первичному ключу \_id

```
db.unicorns.dropIndex('_id_')
```

```
db.unicorns.getIndexes()
< [ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

```
> db.unicorns.dropIndex('_id_')
```

```
✖ ▶ MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

33: Ошибка при попытке удаления обязательного индекса \_id\_]

**Вывод:** Управление индексами позволяет удалять созданные пользователем структуры (например, name\_1), однако дефолтный системный индекс *id* защищен от удаления на уровне ядра СУБД.

## Практическое задание 4.4.1. План запроса и сравнение производительности индекса

### Команды:

JavaScript

```
db.numbers.drop()
```

```
// Генерация 100 000 записей
for (let i = 0; i < 100000; i++) {
  db.numbers.insertOne({value: i})
}
```

```
// 1. План выполнения ДО создания индекса
var noIndex = db.numbers.find({value: {$gte: 99996}}).sort({value: -1}).limit(4).explain('executionStats')
print("Время без индекса: " + noIndex.executionStats.executionTimeMillis + " мс")
```

```
// Создание индекса
db.numbers.createIndex({value: 1})
```

```
// 2. План выполнения ПОСЛЕ создания индекса
var withIndex = db.numbers.find({value: {$gte: 99996}}).sort({value: -1}).limit(4).explain('executionStats')
print("Время с индексом: " + withIndex.executionStats.executionTimeMillis + " мс")
```

```
> var noIndex = db.numbers.find({value: {$gte: 99996}}).sort({value: -1}).limit(4).explain('executionStats')
print("Время без индекса: " + noIndex.executionStats.executionTimeMillis + " мс")
```

```
< Время без индекса: 71 мс
```

34: Параметры executionStats плана без индекса (COLLSCAN)]

```
> db.numbers.createIndex({value: 1})
< value_1
```

35: Создание индекса value\_1 в коллекции numbers]

```
> var withIndex = db.numbers.find({value: {$gte: 99996}}).sort({value: -1}).limit(4).explain('executionStats')
print("Время с индексом: " + withIndex.executionStats.executionTimeMillis + " мс")
```

```
< Время с индексом: 14 мс
```

36: Параметры executionStats плана с индексом (IXSCAN)]

**Вывод:** До оптимизации использовался режим последовательного сканирования всей коллекции (stage: COLLSCAN), затративший 71 мс на перебор всех 100 000 документов. После создания индекса СУБД переключилась на индексный поиск (stage: IXSCAN), время выполнения снизилось до 14 мс, а число исследованных ключей составило всего 4. Это подтверждает ключевую роль индексов в ускорении поиска.

### Выводы

В ходе выполнения лабораторной работы была развернута и исследована база данных learn в актуальной версии СУБД MongoDB. На практике освоены базовые операции манипулирования данными (CRUD) с применением продвинутых функций фильтрации, логических условий и проекций. Были изучены подходы к хранению сложноструктурированных документов, включая вложенные объекты и массивы, а также итерация результатов через курсоры. С помощью методов count, distinct и компонента aggregate решены задачи аналитической группировки данных. В финальной части работы были изучены механизмы ссылочной целостности DBRef и методы оптимизации производительности СУБД через создание уникальных и простых b-tree индексов, эффективность которых подтверждена анализом планов выполнения запросов (explain).

## **Список источников**

1. MongoDB CRUD Operations [Электронный ресурс] // MongoDB Documentation. URL: <https://docs.mongodb.com/manual/crud/>
2. MongoDB Manual [Электронный ресурс] // MongoDB Documentation. URL: <https://docs.mongodb.com/manual/>
3. Онлайн-руководство по MongoDB [Электронный ресурс] // METANIT.COM. URL: <https://metanit.com/nosql/mongodb/>
4. Кайл Б. MongoDB в действии [Электронный ресурс]. Москва: ДМК Пресс.